

# 王道考研——数据结构

WWW.CSKAOYAN.COM

## 第四章 串

本节内容

串

定义  
基本操作

# 知识总览



注：数据结构三要素——逻辑结构、数据的运算、存储结构（物理结构）

存储结构不同，运算的实现方式不同

## 串的定义

串，即字符串（String）是由零个或多个字符组成的有限序列。一般记为

$S = 'a_1a_2\cdots a_n'$  ( $n \geq 0$ )

其中， $S$ 是串名，单引号括起来的字符序列是串的值； $a_i$ 可以是字母、数字或其他字符；串中字符的个数 $n$ 称为串的长度。 $n = 0$ 时的串称为空串（用 $\emptyset$ 表示）。

例：

$S = \text{"HelloWorld!"}$

$T = \text{'iPhone 11 Pro Max?'}$

注：有的地方用双引号（如Java、C）  
有的地方用单引号（如Python）

子串：串中任意个连续的字符组成的子序列。

主串：包含子串的串。

字符在主串中的位置：字符在串中的序号。

子串在主串中的位置：子串的第一个字符在主串中的位置。

Eg: 'iPhone', 'Pro M' 是串T的子串

Eg: T 是子串'iPhone'的主串

Eg: '1'在T中的位置是8(第一次出现)

Eg: '11 Pro'在T中的位置为8

空串 V.S 空格串：

$M = ''$

M是空串

$N = ' '$

N是由三个空格字符组成的空格串，每个空格字符占1B

注意：位序从1开始  
而不是从0开始



## 串 V.S 线性表

串是一种特殊的线性表，数据元素之间呈线性关系



有啥不一样~~

串的数据对象限定为字符集（如中文字符、英文字符、数字字符、标点字符等）

串的基本操作，如增删改查等通常以子串为操作对象



我们不一样

# 串 V.S 线性表



通常以“子串”为增删改查的操作对象



人类的语言通常要多个字符组成的序列才有现实意义

## 串的基本操作

假设有串 $T=""$ ， $S="iPhone 11 Pro Max?"$ ， $W="Pro"$

**StrAssign(&T,chars)**: 赋值操作。把串 $T$ 赋值为 $chars$ 。

**StrCopy(&T,S)**: 复制操作。由串 $S$ 复制得到串 $T$ 。

**StrEmpty(S)**: 判空操作。若 $S$ 为空串，则返回TRUE，否则返回FALSE。

**StrLength(S)**: 求串长。返回串 $S$ 的元素个数。

**ClearString(&S)**: 清空操作。将 $S$ 清为空串。

**DestroyString(&S)**: 销毁串。将串 $S$ 销毁（回收存储空间）。

**Concat(&T,S1,S2)**: 串联接。用 $T$ 返回由 $S1$ 和 $S2$ 联接而成的新串

**SubString(&Sub,S,pos,len)**: 求子串。用 $Sub$ 返回串 $S$ 的第 $pos$ 个字符起长度为 $len$ 的子串。

**Index(S,T)**: 定位操作。若主串 $S$ 中存在与串 $T$ 值相同的子串，则返回它的主串 $S$ 中第一次出现的位置；否则函数值为0。

**StrCompare(S,T)**: 比较操作。若 $S>T$ ，则返回值 $>0$ ；若 $S=T$ ，则返回值 $=0$ ；若 $S<T$ ，则返回值 $<0$ 。

Eg: 执行基本操作 **Concat(&T, S, W)** 后， $T="iPhone 11 Pro Max?Pro"$

执行基本操作 **SubString(&T, S, 4, 6)** 后， $T="one 11"$

执行基本操作 **Index(S, W)** 后，返回值为 11

存储空间扩展？

## 串的比较操作

StrCompare(S,T): 比较操作。若 $S>T$ , 则返回值 $>0$ ; 若 $S=T$ , 则返回值 $=0$ ; 若 $S<T$ , 则返回值 $<0$ 。

A

abandon/ ə'bəndən/ vt. 丢弃; 放弃, 抛弃

aboard/ ə'bo:d/ ad. 在船(车)上; 上船

absolute/ 'æbsəlu:t/ a. 绝对的; 纯粹的

absolutely/ 'æbsəlu:tli/ ad. 完全地; 绝对地

absorb/ əb'sɔ:b/ vt. 吸收; 使专心

abstract/ 'æbstrækt/ n. 摘要

abundant/ ə'bʌndənt/ a. 丰富的; 大量的

abuse/ ə'bjuz, ə'bjus/ vt. 滥用; 虐待 n. 滥用

academic/ ækə'demik/ a. 学院的; 学术的

accelerate/ æk'seləreit/ vt. (使) 加快; 促进

“abandon” < “aboard”

从第一个字符开始往后依次对比,  
先出现更大字符的串就更大

长串的前缀与短串相  
同时, 长串更大

“abstract” < “abstraction”

“abstract” < “abstract ”

只有两个串完全相  
同时, 才相等

“academic” > “abuse”

“academic” = “academic”



# 字符集编码

$$y = f(x)$$

字符集：函数定义域

编码：函数映射规则  $f$

$y$ ：对应的二进制数

任何数据存到计算机中一定是二进制数。

需要确定一个字符和二进制数的对应规则

这就是“编码”

“字符集”：

英文字符——ASCII字符集  
中英文——Unicode字符集

基于同一个字符集，可以有多种编码方案，如：UTF-8，UTF-16

注：采用不同的编码方式，每个字符所占空间不同，考研中只需默认每个字符占1B即可

ASCII 字符代码表 一

| 高四位<br><br><br>低四位 |   | ASCII非打印控制字符 |               |      |     |       |      |    |          |       |        | ASCII 打印字符 |      |     |      |      |    |      |     |      |     |      |                |
|--------------------|---|--------------|---------------|------|-----|-------|------|----|----------|-------|--------|------------|------|-----|------|------|----|------|-----|------|-----|------|----------------|
|                    |   | 0000         |               |      |     |       | 0001 |    |          |       |        | 0010       | 0011 |     | 0100 | 0101 |    | 0110 |     | 0111 |     |      |                |
|                    |   | 0            |               |      |     |       | 1    |    |          |       |        | 2          | 3    |     | 4    | 5    |    | 6    |     | 7    |     |      |                |
|                    |   | +进制          | 字符            | ctrl | 代码  | 字符解释  | +进制  | 字符 | ctrl     | 代码    | 字符解释   | +进制        | 字符   | +进制 | 字符   | +进制  | 字符 | +进制  | 字符  | +进制  | 字符  | ctrl |                |
| 0000               | 0 | 0            | BLANK<br>NULL | ^@   | NUL | 空     | 16   | ▶  | ^P       | DLE   | 数据链路转意 | 32         |      | 48  | 0    | 64   | @  | 80   | P   | 96   | `   | 112  | p              |
| 0001               | 1 | 1            | ☺             | ^A   | SOH | 头标开始  | 17   | ◀  | ^Q       | DC1   | 设备控制 1 | 33         | !    | 49  | 1    | 65   | A  | 81   | Q   | 97   | a   | 113  | q              |
| 0010               | 2 | 2            | ☹             | ^B   | STX | 正文开始  | 18   | ↕  | ^R       | DC2   | 设备控制 2 | 34         | "    | 50  | 2    | 66   | B  | 82   | R   | 98   | b   | 114  | r              |
| 0011               | 3 | 3            | ♥             | ^C   | ETX | 正文结束  | 19   | !! | ^S       | DC3   | 设备控制 3 | 35         | #    | 51  | 3    | 67   | C  | 83   | S   | 99   | c   | 115  | s              |
| 0100               | 4 | 4            | ♦             | ^D   | EOT | 传输结束  | 20   | ¶  | ^T       | DC4   | 设备控制 4 | 36         | \$   | 52  | 4    | 68   | D  | 84   | T   | 100  | d   | 116  | t              |
| 0101               | 5 | 5            | ♣             | ^E   | ENQ | 查询    | 21   | §  | ^U       | NAK   | 反确认    | 37         | %    | 53  | 5    | 69   | E  | 85   | U   | 101  | e   | 117  | u              |
| 0110               | 6 | 6            | ♠             | ^F   | ACK | 确认    | 22   | ■  | ^V       | SYN   | 同步空闲   | 38         | &    | 54  | 6    | 70   | F  | 86   | V   | 102  | f   | 118  | v              |
| 0111               | 7 | 7            | ●             | ^G   | BEL | 震铃    | 23   | ↕  | ^W       | ETB   | 传输块结束  | 39         | '    | 55  | 7    | 71   | G  | 87   | w   | 103  | g   | 119  | w              |
| 1000               | 8 | 8            | ◼             | ^H   | BS  | 退格    | 24   | ↑  | ^X       | CAN   | 取消     | 40         | (    | 56  | 8    | 72   | H  | 88   | X   | 104  | h   | 120  | x              |
| 1001               | 9 | 9            | ○             | ^I   | TAB | 水平制表符 | 25   | ↓  | ^Y       | EM    | 媒体结束   | 41         | )    | 57  | 9    | 73   | I  | 89   | Y   | 105  | i   | 121  | y              |
| 1010               | A | 10           | ◻             | ^J   | LF  | 换行/新行 | 26   | →  | ^Z       | SUB   | 替换     | 42         | *    | 58  | :    | 74   | J  | 90   | Z   | 106  | j   | 122  | z              |
| 1011               | B | 11           | ♂             | ^K   | VT  | 竖直制表符 | 27   | ←  | ^[       | ESC   | 转意     | 43         | +    | 59  | ;    | 75   | K  | 91   | [   | 107  | k   | 123  | {              |
| 1100               | C | 12           | ♀             | ^L   | FF  | 换页/新页 | 28   | └  | ^\<br>FS | 文件分隔符 | 44     | ,          | 60   | <   | 76   | L    | 92 | \    | 108 | l    | 124 |      |                |
| 1101               | D | 13           | ♪             | ^M   | CR  | 回车    | 29   | ↔  | ^]<br>GS | 组分分隔符 | 45     | -          | 61   | =   | 77   | M    | 93 | ]    | 109 | m    | 125 | }    |                |
| 1110               | E | 14           | 🎵             | ^N   | SO  | 移出    | 30   | ▲  | ^6<br>RS | 记录分隔符 | 46     | .          | 62   | >   | 78   | N    | 94 | ^    | 110 | n    | 126 | ~    |                |
| 1111               | F | 15           | ☼             | ^O   | SI  | 移入    | 31   | ▼  | ^-<br>US | 单元分隔符 | 47     | /          | 63   | ?   | 79   | O    | 95 | _    | 111 | o    | 127 | Δ    | ^Back<br>space |

注：表中的ASCII字符可以用：ALT + “小键盘上的数字键”输入



## 拓展：乱码问题

𠂔P u 儀 u 儀 θ u ? ? ? 离 𠂔 烂? 朕@ 嬋端 姝 ?? 旅娥 S 嫗婢 鐵 𠂔 婢 姝 [ 脣總 U 嬌? 燒瑁 yj  
 ■ t3 江 ■ t2 ? ? ? ? ? ? 姝 ■ 李 走 ■ 李 - t ■ ? ? ? 粦 D? u ■ ? ? ? ? 儀 ■ u - ? 婢 ^ [ 朕@ SUWQ 嫗場 𠂔 ■ ? \$ 峯 ■ ?  
 [ 脣卸 @ 姝 ■ 肱 SU 婦 婢 ■ 伊 t ] 儻 ■ u W 儻 θ u9 客 ■ R j TP 鑄? yj 幼 ■ 嫩 尅 ■ ( 嫗 瑄 嫗 ■ 瑄 ■ f 并 ■ f  
 𠂔 鐳? yj 肱 姝 ■ 儀 ■ u 嫗 ■ 峙 ■ 胚 U 嫗 ■ { ■ t ■ { ■ u3 嬌 婢 ■ 伊 t 鋪? yj 婢 ■ P 輕? yj3 線 F ■ 嫗 璽 yyy 嫗 璧  
 @ d j ō d? 媼 鳴 x ■ 媼 鳴 p ■ 媼 ■ 媼 鳴 饋 z ■ ? 塘 媼 馱 x, t ■ 媼 鳴 @ ■ 媼 鳴 B ■ 3 縊 Y Y d? ? 環? yj 媼 間  
 ?? yj3 縊 Y Y d? h 謳 h? B 桡? yj 瞄 ■ yj 腩 媼 @ ^ [ Y Y? 弄 SUW 嫗 ▼ 趨 饗 哇 ■ yj : ? 劫 嫗 ! ?  
 ; G ■ 劍 嬌 鑽? yj 嬌 桡? yj 岫 P 嫗 ■ 嫗 ■ 嬌 瑾? yj 膜 媼 ■ 伊 t 𠂔 G u c 嬌? yj U 咸 ■ 近 \$  
 ? yj ■ < \$ t 𠂔 嫗 ■ ? 岳 \$ 𠂔 嫗 嫗 嬌? yj 嫗 X \_ ^ [ 脣總 UW 嫗 獅 鸛 饗 ^ ; < t / 嬌 机? yj U 現 ■ 近 \$ ■ ? 塘 ^ 壘  
 𠂔 嫗 , s ■ ? 嬌? yj Q 軾 岨? 珊 T ? ■ yj 嫗 u 猓 ■ 晤 𠂔 塘 ^ 拖 3 羈 U 橫 E? 媼 玢 U 脣 U 瘡 U  
 < j 鑄? yj 嫗 S 鑿? yj 荒 E? S j ?? yj ? 苔? 苔 苔 豚 ■ 苔? 3 繳 h 盪 d j ō d? 岨 嬌? yj 嫩  
 yj 朕 ^ [ 嬌 ] 朕@ SUW 嫗 獅 鸛 饗 ^ ; < ■ t / 嬌 錫? yj U 現 ■ 近 \$ ■ ? 塘 ^ 墊 \$ 塾 \$ T 娶 ■ 嫗 ■ 嬌 鏡? yj 嫗  
 嫗 PR 宙 \$ ■ 岳 \$ ■ 玛? yj 婦 \$ ■ 尅 ■ 也 \$ 岨 \$ 3 珊 鋁 } yj f 菱 \$ ■ BM 嬌 杪? yj ■ 侯 钲? yj 婢 ■ P 婢 ■ 妹  
 ? yj 婢 P 伊 u ■ ; 鳩 雲? u3 儻 ■ t - ■ { l u' 岳 \$ ■ ? 婢 ■ 桃? yj 雲 f 儻 > ■ u 娶 拎 一 也 \$ ! 也 \$ 婦 \$ 港 \$ 婦  
 𠂔 P 伊 t ; 鳩 勘 P 岳 \$ ■ ? 嫗 璨? yj 客 θ? 嫗 瑁? yj f 儻 > ■ u 鯨 @ t ■ 客 X? 嫗 鑄? yj3 禧 \$ 博 ■  
 桉? yj 嫗 j ZS 鎭? yj X ■ B S j 鑽? yj ? 傅 ? 枋? yj P ■ B [ 朕@ U 娶 3 繳 h 琳 d j ō d? yj 𠂔 B u3 ? 傅 杞 ■  
 % 琳 瞄 𠂔 yj 膝 ] 肱 ?? B 𠂔 錫 yj y h? B 媛? yj h ■ B 钰? yj j 瑙? yj \ ■ B j 鈺? yj \ ■ B j 鑿? yj d ■ B ?  
 嬌 3 務 予 ■ < yj 嬌 ^ [ 朕@ SUWU 嫗 雲 葵 鑑? yj 嫗 ? C ; 航 ■ | yj u ? 詠 ■ 嫗 槩? yj ? N ■ | 7 j u 廬 嫗 + 蔞 嫗  
 U 嫗 嫗 嬌 栳? yj P 嫗 栳? yj P 壁? yj 湮 𠂔 繞 ^ [ 胚 U 嫗 嫗 嬌 嫗 鑑? yj ^ [ 脣? 膏 ■ 續 ■ 髻? ? AN? 霉 ■  
 u 鬚 ■ 狀 發 波 = 1 貢 1 詣? yj 夠 x ?? yj 嫗 ■ ^ 朕@ 漂 u - 1 襌 發 波 Q? 鏤 yyy 夠 x 桉? yj 嫗 ^ 朕@ U 娶 世  
 琳 參 yj P 媛? yj 岨 廢 岨 广 岨 鬚 鑿? yj 伊 u ■ 苔? yj yj 媼 鵲 嫗 ] 胚 嫗 嫗 鑿 yyy @ 璋 [ 朕 娶 3 繳 h 機 @ d j ō d? i

在你的文件中，原本采用某一套编码规则 $y=f(x)$ ，如： ‘码’ 0001010100010101010010

打开文件时，你的软件以为你采用的是另一套编码规则 $y=g(x)$ ，如： 0001010100010101010010

# 知识回顾与重要考点

## 串

### 定义

串，即字符串（String）是由零个或多个字符组成的有限序列

术语：串长、空串、空格串、子串、主串、字符在主串中的位置、子串在主串中的位置

### 串V.S线性表

串的数据对象限定为字符集

串的基本操作大多以“子串”为操作对象

### 基本操作

$\text{Index}(S, T)$ ，定位操作，找到串 $T$ 在主串 $S$ 中的位置

$\text{StrCompare}(S, T)$ ：比较操作。若 $S > T$ ，则返回值 $> 0$ ；若 $S = T$ ，则返回值 $= 0$ ；若 $S < T$ ，则返回值 $< 0$ 。

其他...

想想四六级词汇表如何排序

### 字符集编码

每个字符在计算机中对应一个二进制数，比较字符的大小其实就是比较二进制数的大小

本节内容

串

存储结构



## 知识总览

### 串的存储结构

顺序存储

链式存储

基于顺序存储实现基本操作

## 串的顺序存储

结合顺序表的知识思考优缺点

```
#define MAXLEN 255
```

```
typedef struct{
```

```
    char ch[MAXLEN];
```

```
    int length;
```

```
}SString;
```

静态数组实现  
(定长顺序存储)

//预定义最大串长为255

//每个分量存储一个字符

//串的实际长度

```
typedef struct{
```

```
    char *ch;
```

```
    int length;
```

```
}HString;
```

动态数组实现  
(堆分配存储)

//按串长分配存储区, ch指向串的基地址

//串的长度

```
HString S;
```

```
S.ch = (char *) malloc(MAXLEN * sizeof(char));
```

```
S.length = 0;
```

用完需要手动free

S.ch →

内存

ch[0]

ch[1]

ch[2]

ch[3]

ch[4]

...

...

ch[254]

length

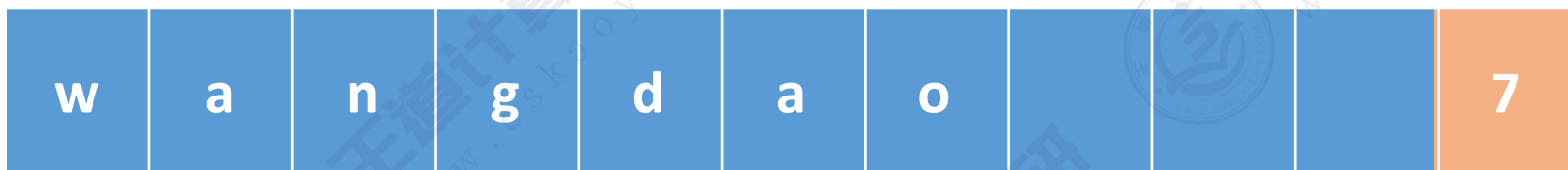
分配连续的存储空间,  
每个 char 字符占 1B

串的实际长度

## 串的顺序存储

char ch[10]

方案一:



变量Length

ch[0]充当 Length

方案二:



优点: 字符的位序  
和数组下标相同

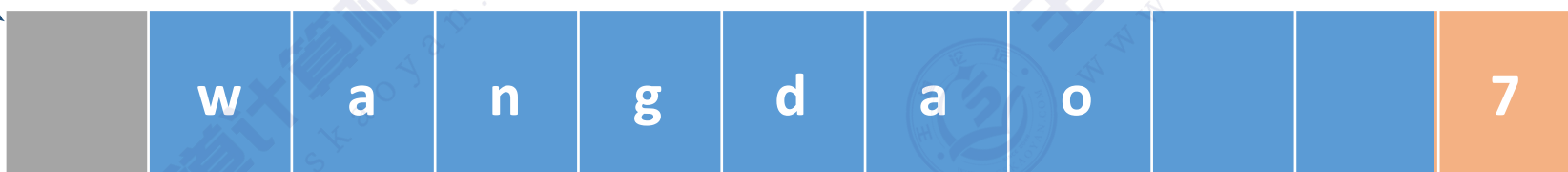
方案三:



没有Length变量,  
以字符'\0'表示结尾  
(对应ASCII码的0)

ch[0]废  
弃不用

方案四:  
(教材)



变量Length

## 串的链式存储

结合链表的知识  
思考优缺点

```
typedef struct StringNode{  
    char ch;    //每个结点存1个字符  
    struct StringNode * next;  
}StringNode, * String;
```



存储密度低：每个字  
符1B，每个指针4B

```
typedef struct StringNode{  
    char ch[4];    //每个结点存多个字符  
    struct StringNode * next;  
}StringNode, * String;
```



存储密度  
提高



## 基本操作的实现

ch[0]废弃不用

方案四：  
(教材)



变量Length

```
#define MAXLEN 255           //预定义最大串长为255
typedef struct{
    char ch[MAXLEN];         //每个分量存储一个字符
    int length;               //串的实际长度
}SString;
```

StrAssign(&T,chars): 赋值操作。把串T赋值为chars。

StrCopy(&T,S): 复制操作。由串S复制得到串T。

StrEmpty(S): 判空操作。若S为空串，则返回TRUE，否则返回FALSE。

StrLength(S): 求串长。返回串S的元素个数。

ClearString(&S): 清空操作。将S清为空串。

DestroyString(&S): 销毁串。将串S销毁（回收存储空间）。

Concat(&T,S1,S2): 串联接。用T返回由S1和S2联接而成的新串

## 基本操作的实现

**SubString(&Sub,S,pos,len)**: 求子串。用Sub返回串S的第pos个字符起长度为len的子串。

S.ch="wangdao"  
S.length=7

|   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|
| S |   | w | a | n | g | d | a | o |   |   |
|   | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |

//求子串

```
bool SubString(SString &Sub, SString S, int pos, int len){
```

```
    //子串范围越界
```

```
    if (pos+len-1 > S.length)
```

```
        return false;
```

```
    for (int i=pos; i<pos+len; i++)
```

```
        Sub.ch[i-pos+1] = S.ch[i];
```

```
    Sub.length = len;
```

```
    return true;
```

```
}
```

```
#define MAXLEN 255
```

```
typedef struct{
```

```
    char ch[MAXLEN];
```

```
    int length;
```

```
}SString;
```

//预定义最大串长为255

//每个分量存储一个字符

//串的实际长度

## 基本操作的实现

**StrCompare(S,T)**: 比较操作。若 $S>T$ , 则返回值 $>0$ ; 若 $S=T$ , 则返回值 $=0$ ; 若 $S<T$ , 则返回值 $<0$ 。

S.ch="wangdao"  
S.length=7

S.ch →

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
|   | w | a | n | g | d | a | o |   |   |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |

//比较操作。若 $S>T$ , 则返回值 $>0$ ; 若 $S=T$ , 则返回值 $=0$ ; 若 $S<T$ , 则返回值 $<0$

```
int StrCompare(SString S, SString T) {  
    for (int i=1; i<=S.length && i<=T.length; i++){  
        if (S.ch[i]!=T.ch[i])  
            return S.ch[i]-T.ch[i];  
    }  
    //扫描过的所有字符都相同, 则长度长的串更大  
    return S.length-T.length;  
}
```

T1

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
|   | w | b | n | g | d | a | o |   |   |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |

T2

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
|   | w | a | n | g |   |   |   |   |   |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |

## 基本操作的实现

**Index(S,T):** 定位操作。若主串S中存在与串T值相同的子串，则返回它的主串S中第一次出现的位置；否则函数值为0。

S.ch="wangdao"  
S.length=7

S.ch →

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
|   | w | a | n | g | d | a | o |   |   |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |

```
int Index(SString S, SString T){  
    int i=1, n=StrLength(S), m=StrLength(T);  
    SString sub;    //用于暂存子串  
    while(i<=n-m+1){  
        SubString(sub, S, i, m);  
        if(StrCompare(sub, T)!=0) ++i;  
        else return i; //返回子串在主串中的位置  
    }  
    return 0; //S中不存在与T相等的子串  
}
```

T

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
|   | g | d | a |   |   |   |   |   |   |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |



# 知识回顾与重要考点

## 串的存储结构

顺序存储

静态数组

```
#define MAXLEN 255
```

//预定义最大串长为255

```
typedef struct{
```

```
    char ch[MAXLEN];
```

//每个分量存储一个字符

```
    int length;
```

//串的实际长度

```
}SString;
```

动态数组

malloc、free

链式存储

可让每个结点存多个字符，没有字符的位置用'#'或'\0'补足

王道教材采用  
——静态数组

ch[0] 废弃不用

方案四：  
(教材)

w a n g d a o

7

变量Length

基本操作的实现

求子串: bool SubString(SString &Sub, SString S, int pos, int len)

串的比较: int StrCompare(SString S, SString T)

求串在主串中的位置: int Index(SString S, SString T)